

Refactoring to Patterns

CSE301

University of Sunderland

Harry R Erwin, PhD

Introduction

- Want to use a pattern in an existing programme, but don't know how to do it?
- Don't panic—this lecture will discuss how to refactor a programme to introduce a pattern.
- Resource:
 - Kerievsky, J., 2005, *Refactoring to Patterns*, Addison-Wesley.

Topics

- Creation—creation methods, factory, builder, and singleton
- Simplification—composition, strategy, decorator, state, composite, command
- Generalization—template, composite, observer, adapter, interpreter
- Protection—singleton, null object, generalization
- Accumulation (visitor) patterns—will be skipped.
- Miscellany—utilities

Creation Patterns

- These refactorings address some of the common problems in this area.
- The Creation Method pattern is new. It describes the replacement of a collection of constructors with static methods that do the same thing more clearly.
- Factory classes usually implement one or more Creation Methods.

“Replace Constructors with Creation Methods”

- Multiple constructors for a class make it hard to choose the right one.
- Solution: replace the constructors with static (or non-static) creation methods that are easier to understand. Make the constructors private.
- These are often called factory methods.
- It does make object creation for that class *non-standardised*, since the **new** operator is not available.

“Move Creation Knowledge to Factory”

- Data and code to instantiate a class is sprawled across many classes.
- Move the creation knowledge into a single Factory class to encapsulate the creation logic and client preferences. Start by writing a creation method. Then create a Factory class and move the creation method into it. Chase down all the compilation problems that result. You’re done.
- Avoid *overuse* of the Factory pattern. That will overcomplicate your design.

“Encapsulate Classes With Factory”

- Clients directly instantiate classes that reside in one package and implement a common interface.
- Solution: Make the class constructors non-public and introduce a Factory class to create them. (See `java.util.Collections`.) Improves encapsulation.
- First use Extract Method on constructor calls and then Move Method to move them to the Factory class. Make the constructors non-public.
- Negatives: new creation methods for new kinds of instances. Also limits client customization as source code is unavailable to clients.

“Inline Singleton”

- Too many globals are bad, so you’ve become addicted to Singleton classes. Your code needs access to a unique object, but *doesn’t* need a global point of access. This is a *normal* situation.
- Solution: Move the Singleton’s features to a class that stores and provides access to the object. Delete the Singleton.
- Use Move Method and Move Field to do this.
- This does *complicate* a design when objects need to be passed down chains.

Simplification

- Most of your code starts out complicated.
The goal of these refactorings is to redesign it for simplicity.

“Compose Method”

- You can't understand a method's logic.
- Solution: Transform the logic into a small number of intention-revealing steps at the same level of detail. Will communicate and simplify what the method does.
- I use Extract Method heavily to do this, particularly for GUIs.
- Leads to many small methods of 5-10 lines. Can make debugging a chore.

“Replace Conditional Logic with Strategy”

- Sign: lots of conditional logic in a method.
- Solution: Create a Strategy for each variant and delegate the computing to the Strategy instance. The Strategies encapsulate the various possible conditions. Think about using the Prototype pattern to avoid creation overhead.
- It *can* complicate a design or algorithm.

“Move Embellishment to Decorator”

- Code embellishes a class’s core responsibility.
- Solution: Move it to a Decorator. Note that the Decorator class has to implement the public interface of the decorated class, so you may want to refactor towards this, rather than all the way.
- First create the Decorator class (mucho work). Then find where the choice of embellishment is made and instead construct a Decorator around the base class. Note Decorators may be constructed around Decorators.

“Replace State-Altering Conditionals with State”

- The conditional expressions controlling an object's state are complex.
- Solution: Replace the conditionals with State classes that represent specific states.
- Find where the state fields are maintained and encapsulate them in an abstract State class. Use Extract Subclass to get concrete States. Use Move Method to delegate responsibilities to the subclasses. Identify state transitions and switch the State instance there. Consider using Prototype.
- NB: State and Strategy patterns are different!

“Replace Conditional Dispatcher with Command”

- Conditional logic (if then else or switch) is used to dispatch requests and execute actions.
- Solution: Create a command for each action. Store in a collection and replace the conditional logic with code to fetch and execute commands.
- This is done using Extract Method over and over again. Then you Extract Class on each method. Create a Command interface and an execute() method for that interface. Build a CommandMap. Finally replace the conditional dispatcher with logic that selects the Command from the CommandMap and executes it.

Generalization

- Converts specific code to general-purpose code.
- Removes duplicated code.
- Simplifies and clarifies code.

“Form Template Method”

- Two methods in subclasses perform similar but not identical steps in the same order.
- Solution: Generalize the two methods by extracting their steps into methods with identical signatures. Use Extract Method to do this.
- Then pull up the generalized methods to form a template method in their base class.

“Replace Hard-Coded Notifications with Observer”

- Subclasses are defined and hard-coded to notify an instance of another class of their changes. The notified class varies with the subclass selected.
- Solution: Remove the subclasses by making their superclass capable of notifying any class that implements an Observer pattern. The receiver implements that and registers for updates.
- You may need to move logic from the subclass to its receiver to allow this to work.

“Unify Interfaces with Adapter”

- Clients interact with two alternative classes, one of which has a preferred or mandatory interface.
- Solution: Unify the interfaces with an adapter.
- Apply Extract Interface to the class with the preferred interface.
- Apply Extract Class to the class that uses the alternatives. This creates a primitive Adapter.
- Use the Adapter instead of the hidden classes. Use Move Method to make this happen.
- Have the Adapter ‘implement’ the common interface.

Protection

- These refactorings improve the protection of existing code.

“Replace Type Code with Class”

- A field's type (primitive type or String) does not protect it from unsafe assignments and invalid equality comparisons. This can be a problem for 'enumerations'.
- Solution: Make the type of the field a class so you can constrain those unsafe operations.
- Apply Self Encapsulate Field.
- Create a concrete class to hold the field.
- Instantiate in the using class and switch over.

“Limit Instantiation with Singleton”

- Your code creates multiple instances of an object and then uses too much memory or slows down.
- Solution: Replace the multiple instances with a Singleton
- Make sure the state of the object can be shared.
- Replace the constructor with a creation method. Then insert the standard Singleton logic.
- You also might work with a Map if you still need more than one object.

“Introduce Null Object”

- Your code has lots of logic for dealing with a null field or variable.
- Solution: Replace the null value with a Null Object, which has all the necessary methods defined and simply does nothing.
- Create a NullObject class by applying Extract Subclass and/or Extract Interface.
- Find out what the null checks do in your code and have the NullObject class do them.
- Where the field is set to null, set it to an instance of the NullObject.
- Eventually, you will remove the null checks.

Miscellany

- Low-level refactorings
 - Chain Constructor
 - Unify Interface
 - Extract Parameter

“Chain Constructor”

- You have multiple constructors with duplicate code.
- Solution: Chain them together
- Look for duplication and figure out ways that the constructors can call each other to eliminate it.
- Make any constructors that are internal to this process non-public

“Unify Interface”

- You need a superclass or superinterface to have the same interface as a subclass.
- Solution: Find all public methods in the subclass and put null versions in the superclass.
- Often a station on a road to someplace else.

“Extract Parameter”

- A method or constructor assigns a field to a locally instantiated value.
- Solution: Assign the field to a parameter supplied by a client by extracting one-half of the assignment statement to a parameter.
- The client supplies a null variable to fill in, and the constructor or method sets it to the value.
- It's like a website cookie. If the variable is final, this can even ensure that it is preserved unchanged until it is needed later.